Guía de la Prueba de desempeño – Módulo 6

Arquitectura Hexagonal + JPA Avanzado + Seguridad JWT + Observabilidad + Microservicios + Docker + Pruebas

Nombre de la prueba: HelpFlow – Sistema Integral de Gestión de Soporte TI
Módulo de formación al que corresponde: Módulo 6 – Java + Spring Boot
Duración de la prueba: La prueba tendrá un calendario semanal con la siguiente estructura para las jornadas AM y PM:

Hora	Día 1	Día 2	Día 3	Día 4	Día 5
06:00 – 07:00	Prueba técnica				
07:00 – 08:00					
08:00 – 08:20	Break 1	Sustentación de la solución entregada (individual)* *Con previa cita asignada*	Sustentación de la solución entregada (individual)* *Con previa cita asignada*	Retroalimentación personalizada por parte de tu TL* *Con previa cita asignada*	Carga de resultados y retroalimentación a Moodle por parte de tu TL*
08:20 – 09:00	Prueba técnica				
09:00 – 10:00					
10:00 – 10:20	Break 2				
10:20 – 11:00	Prueba técnica				
11:00 – 12:00					
12:00 – 12:20	Break 3				
12:20 – 13:00	Prueba técnica				
13:00 – 14:00					
Cambio de jornada					
14:00 – 15:00	Prueba técnica				
15:00 – 16:00					
16:00 – 16:20	Break 1	Sustentación de la solución entregada (individual)* *Con previa cita asignada*	Sustentación de la solución entregada (individual)* *Con previa cita asignada*	Feedback personalizado por parte de tu TL* *Con previa cita asignada*	Carga de resultados y retroalimentación a Moodle por parte de tu TL*
16:20 – 17:00	Prueba técnica				
17:00 – 18:00					
18:00 – 18:20	Break 2				
18:20 – 19:00	Prueba técnica				
19:00 – 20:00					
20:00 – 20:20	Break 3				
20:20 – 21:00	Prueba técnica				
21:00 – 22:00					

Notas: **Durante **toda la jornada** tendrás trabajo **individual** asignado en el cual deberás estar avanzando, mientras eres llamado a tu cita de sustentación o feedback en la sala de entrenamiento habitual. Durante el **día 5** tu TL no estará disponible para resolver dudas, pero cuentas con tu tutor oficial.*

Temas abordados:

- Arquitectura Hexagonal aplicada al dominio de soporte TI
- Spring Boot (REST, validaciones, AOP, ControllerAdvice)
- JPA + Hibernate avanzado (relaciones, transacciones, EntityGraph)

- Seguridad con Spring Security + JWT
- MapStruct para mapeo
- Observabilidad: Actuator + Micrometer + logging estructurado
- Pruebas (JUnit, Mockito, Spring Boot Test)
- Testcontainers
- Microservicios de tickets y despacho técnico con comunicación REST
- Docker y docker-compose
- Flyway para migraciones

Herramientas y recursos:

Hardware requerido:
Laptop o computadora con conexión a internet (otorgada por Riwi)

Software requerido:

- IntelliJ IDEA o VS Code con extensiones de Java
- JDK 17 o superior
- Maven
- Spring Boot 3+
- PostgreSQL o MySQL
- Docker + Docker Compose
- Postman o Insomnia
- Git y GitHub
- Lombok
- MapStruct
- JUnit 5, Mockito
- Testcontainers

Recursos adicionales:
Documentación oficial de Spring Boot, Spring Security, JPA, Micrometer y Docker.

Apuntes, ejercicios y guías del módulo 6.

Objetivo de la prueba:

Construir un sistema profesional compuesto por dos microservicios para registrar, priorizar, asignar, escalar y cerrar tickets de soporte TI, aplicando arquitectura hexagonal, persistencia avanzada, seguridad JWT, observabilidad, pruebas y despliegue con Docker.

La prueba integra todas las competencias desarrolladas durante las semanas 2–6 y busca identificar tu nivel de dominio en:

- Arquitectura hexagonal con dominio puro, puertos y adaptadores.
- Dos microservicios comunicándose mediante REST.

- Seguridad stateless con JWT y control de acceso por roles.
- Persistencia JPA avanzada, transacciones y optimización de consultas.
- Observabilidad, métricas y logging estructurado.
- Pruebas unitarias, de integración y con Testcontainers.
- Arquitectura hexagonal con dominio puro, puertos y adaptadores.
- Dos microservicios comunicándose mediante REST.
- Seguridad stateless con JWT y control de acceso por roles.
- Persistencia JPA avanzada, transacciones y optimización de consultas.
- Observabilidad, métricas y logging estructurado.
- Pruebas unitarias, de integración y con Testcontainers.
- Contenerización completa y documentación profesional.

El objetivo es construir:

1. support-ticket-service

Servicio central encargado de:

- Gestión de usuarios, técnicos y perfiles de soporte.
- Registro, priorización, asignación, escalamiento y cierre de tickets.
- Integración con technician-dispatch-service.
- Control de roles y seguridad JWT.
- Gestión de usuarios, técnicos y perfiles de soporte.
- Registro, priorización, asignación, escalamiento y cierre de tickets.
- Integración con technician-dispatch-service.
- Control de roles y seguridad JWT.
- Historial, SLA, errores estandarizados, observabilidad y métricas.

2. technician-dispatch-service

Servicio externo simulado que devuelve un técnico disponible y un tiempo estimado de atención según la categoría, prioridad y sede del ticket.

Estructura de la prueba:

Esta prueba plantea un caso de uso real en el que deberás resolver las necesidades de HelpFlow, simulando un entorno profesional y distribuido.

Caso de uso:
HelpFlow gestiona solicitudes de soporte mediante correos, mensajes y hojas de cálculo. Esto ocasiona tickets duplicados o extraviados, asignaciones manuales, falta de trazabilidad, incumplimiento de SLA, controles de acceso insuficientes y capacidad limitada de pruebas, métricas y monitoreo. La empresa requiere un sistema modular, seguro y escalable para registrar, priorizar, asignar, escalar y cerrar tickets.

Parte 1 – Análisis, diseño y gestión de usuarios y técnicos

- Modelar Usuario, Técnico, Ticket, Historial y Asignación.
- Gestionar documento, nombre, correo corporativo, sede, área, especialidad, perfil de soporte y estado ACTIVO o INACTIVO.
- Garantizar documento y correo únicos, correo corporativo válido y usuario ACTIVO para registrar tickets.
- Diseñar diagramas de arquitectura hexagonal, ciclo de vida del ticket y relación entre microservicios.
- Modelar Usuario, Técnico, Ticket, Historial y Asignación.
- Gestionar documento, nombre, correo corporativo, sede, área, especialidad, perfil de soporte y estado ACTIVO o INACTIVO.
- Garantizar documento y correo únicos, correo corporativo válido y usuario ACTIVO para registrar tickets.
- Diseñar diagramas de arquitectura hexagonal, ciclo de vida del ticket y relación entre microservicios.
- Definir puertos de repositorio, autenticación y TechnicianDispatchPort, junto con los casos de uso del dominio.

Parte 2 – Gestión de tickets, dominio y persistencia JPA

- Cada ticket debe registrar solicitante, título, descripción, categoría, prioridad, sede, activo afectado, fechas de creación y límite de SLA, estado y técnico asignado.
- Categorías: HARDWARE, SOFTWARE, RED, ACCESOS y OTRO. Prioridades: BAJA, MEDIA, ALTA y CRÍTICA.
- Estados: ABIERTO, ASIGNADO, EN_PROGRESO, RESUELTO, CERRADO y CANCELADO.
- Calcular el SLA, consultar el servicio de despacho y asignar según especialidad, carga, prioridad, sede y cumplimiento del SLA.
- Registrar historial, validar transiciones y escalar automáticamente tickets próximos a incumplir el SLA.
- Implementar Usuario 1–N Tickets, Ticket 1–N Historiales y Ticket N–1 Técnico; evitar N+1 con EntityGraph, join fetch o batch-size.
- Ejecutar asignación, historial y escalamiento dentro de transacciones.
- Cada ticket debe registrar solicitante, título, descripción, categoría, prioridad, sede, activo afectado, fechas de creación y límite de SLA, estado y técnico asignado.
- Categorías: HARDWARE, SOFTWARE, RED, ACCESOS y OTRO. Prioridades: BAJA, MEDIA, ALTA y CRÍTICA.
- Estados: ABIERTO, ASIGNADO, EN_PROGRESO, RESUELTO, CERRADO y CANCELADO.
- Calcular el SLA, consultar el servicio de despacho y asignar según especialidad, carga, prioridad, sede y cumplimiento del SLA.
- Registrar historial, validar transiciones y escalar automáticamente tickets próximos a incumplir el SLA.

- Implementar Usuario 1–N Tickets, Ticket 1–N Historiales y Ticket N–1 Técnico; evitar N+1 con EntityGraph, join fetch o batch-size.
- Ejecutar asignación, historial y escalamiento dentro de transacciones.
- Usar Flyway: V1__schema, V2__asignaciones_historial_sla y V3__datos_iniciales (opcional).

Parte 3 – Seguridad, validaciones y manejo global de errores

- Implementar autenticación JWT stateless y contraseñas cifradas con PasswordEncoder.
- Exponer /auth/register y /auth/login.
- Roles: ROLE_USUARIO, ROLE_TECNICO y ROLE_ADMIN.
- Usuario: crear y consultar únicamente sus tickets; Técnico: consultar y gestionar los asignados; Admin: acceso completo, reasignación y configuración de SLA.
- Aplicar Bean Validation y validaciones cruzadas para transición de estado, prioridad, técnico habilitado y fecha límite de SLA.
- Usar @ControllerAdvice y ProblemDetail con type, title, status, detail, instance, timestamp y traceId.
- Implementar autenticación JWT stateless y contraseñas cifradas con PasswordEncoder.
- Exponer /auth/register y /auth/login.
- Roles: ROLE_USUARIO, ROLE_TECNICO y ROLE_ADMIN.
- Usuario: crear y consultar únicamente sus tickets; Técnico: consultar y gestionar los asignados; Admin: acceso completo, reasignación y configuración de SLA.
- Aplicar Bean Validation y validaciones cruzadas para transición de estado, prioridad, técnico habilitado y fecha límite de SLA.
- Usar @ControllerAdvice y ProblemDetail con type, title, status, detail, instance, timestamp y traceId.
- Personalizar errores JPA, acceso denegado y validaciones; implementar logging estructurado.

Parte 4 – Microservicios, integración y observabilidad

- Construir technician-dispatch-service sin JPA ni seguridad.
- Exponer POST /technician-dispatch con ticketId, categoria y prioridad.
- Generar una respuesta determinista: la misma combinación debe devolver el mismo técnico sugerido y tiempo estimado.
- Clasificar disponibilidad: 0–2 horas ALTA, 3–8 horas MEDIA y más de 8 horas BAJA.
- Integrar desde support-ticket-service mediante un adaptador REST.
- Exponer Actuator: /health, /info, /metrics y, opcionalmente, /prometheus.
- Construir technician-dispatch-service sin JPA ni seguridad.
- Exponer POST /technician-dispatch con ticketId, categoria y prioridad.
- Generar una respuesta determinista: la misma combinación debe devolver el mismo técnico sugerido y tiempo estimado.
- Clasificar disponibilidad: 0–2 horas ALTA, 3–8 horas MEDIA y más de 8 horas BAJA.
- Integrar desde support-ticket-service mediante un adaptador REST.
- Exponer Actuator: /health, /info, /metrics y, opcionalmente, /prometheus.

- Medir tiempo medio de primera respuesta y resolución, tickets vencidos por SLA, tickets por categoría y prioridad, fallas de autenticación y accesos denegados.

Parte 5 – Pruebas, Docker y entrega final

- Pruebas unitarias con JUnit y Mockito para los casos de uso puros y mock de TechnicianDispatchPort.
- Pruebas de integración con Spring Boot Test y MockMvc para CRUD, asignación, transiciones y seguridad por propietario o técnico.
- Testcontainers con base de datos en contenedor y pruebas reproducibles.
- Dockerfile multi-stage: build con Maven y JDK; ejecución con JRE slim.
- Pruebas unitarias con JUnit y Mockito para los casos de uso puros y mock de TechnicianDispatchPort.
- Pruebas de integración con Spring Boot Test y MockMvc para CRUD, asignación, transiciones y seguridad por propietario o técnico.
- Testcontainers con base de datos en contenedor y pruebas reproducibles.
- Dockerfile multi-stage: build con Maven y JDK; ejecución con JRE slim.
- docker-compose con support-ticket-service, technician-dispatch-service y db; gateway o config-server opcionales.

- README completo con descripción, endpoints, ejecución local y con docker-compose, roles, flujo de asignación, transiciones, reglas de SLA y capturas de pruebas.
- Colección de pruebas Postman u otra, documentación Swagger y diagramas requeridos.
- README completo con descripción, endpoints, ejecución local y con docker-compose, roles, flujo de asignación, transiciones, reglas de SLA y capturas de pruebas.
- Colección de pruebas Postman u otra, documentación Swagger y diagramas requeridos.
- Evidencia de métricas, tickets vencidos y logs.

Permisos y limitaciones:

Se permite:

- Uso de documentación oficial (Spring, JPA, Security, Docker)
- Consulta de tus apuntes, ejercicios o materiales del módulo.
- Uso de IA bajo los lineamientos establecidos y aprobados por el TL.
- Uso de herramientas profesionales (IntelliJ, Docker, Postman)

No se permite:

- Comunicación con otros estudiantes durante la prueba.
- Copiar código o soluciones de terceros.
- Subir el código a repositorios externos durante la evaluación.

Integridad académica:
Cualquier forma de copia, plagio o uso de material no autorizado resultará en **calificación cero** y sanciones según las políticas internas de Riwi.

Idioma:
Todo el código, nombres de variables, clases, métodos y el README deben estar escritos en **inglés**. Solo los datos mostrados en interfaz o mensajes al usuario pueden estar en español.

Implicaciones:

- Esta prueba determina tu continuidad en el programa de entrenamiento según el Capítulo IV - Evaluación y calificación del Reglamento Interno de Riwi.
- Se evaluará tanto la solución técnica como tu defensa en la sustentación individual.

Evaluación:

Rúbrica: disponible en Moodle.
Se evaluarán los siguientes criterios:

- Arquitectura hexagonal pura y dominio sin dependencias de frameworks.
- Microservicios e integración de despacho funcionando.
- Seguridad JWT, roles, propiedad de tickets y accesos restringidos.
- Validaciones, ProblemDetail y logging estructurado.
- Persistencia JPA, MapStruct y procesos transaccionales.
- Pruebas unitarias, integración y Testcontainers.
- Actuator y métricas clave operativas.
- Dockerfile y docker-compose funcionales.
- Arquitectura hexagonal pura y dominio sin dependencias de frameworks.
- Microservicios e integración de despacho funcionando.
- Seguridad JWT, roles, propiedad de tickets y accesos restringidos.
- Validaciones, ProblemDetail y logging estructurado.
- Persistencia JPA, MapStruct y procesos transaccionales.
- Pruebas unitarias, integración y Testcontainers.
- Actuator y métricas clave operativas.
- Dockerfile y docker-compose funcionales.
- README, Swagger, colección de pruebas, diagramas y evidencias.

Entrega:
Mediante la plataforma Moodle en el espacio asignado. Deberás entregar el enlace a un repositorio GitHub público, el proyecto comprimido (.zip), la colección de pruebas con Swagger, el README, los diagramas y las evidencias de métricas, tickets vencidos y logs.

Visualización de resultados:
En Moodle al final del quinto día.

Lugar de retroalimentación:
Sala habitual de entrenamiento durante el cuarto día.

¡Éxitos en este evento evaluativo!